

IPChanger & the Network Configuration Operators problem

Everything we tried to let a non-admin field tech change a PC's IP on a hardened Windows 11 box — why most of it failed, what actually works, and the one registry value that was the real culprit.

OBJECTIVE

Non-admin IPv4 change for gear provisioning

TEST RIG

W104 · Win11 25H2 (26200) · MS hardening baseline

IDENTITIES

winston (user) · x_Winston (xID → NetOps → NCO)

STATUS

Root cause found · Resolved

Effort summary

Approximate hours, reconstructed from the working log and spread across 29–31 Jul 2026 (plus the earlier IPChanger work). Dead ends and rejected approaches are counted deliberately — the original self-elevating app, the DHCP build that was ultimately set aside, and the late-night lab firefighting are where most of the real time actually went. "Just use a tool" quietly erases all of it.

PHASE	WORK	EST. HRS
Original IPChanger	Self-elevating WinForms app — the first approach, which ultimately didn't fit (RunAs elevation, credential-relaunch loop)	5.0
Investigation	NCO + UAC token filtering by logon type; the interactive-vs-network discovery; capability matrix	6.0
Total (estimated)		45.0

PHASE	WORK	EST. HRS
Proof	Control experiments — NCO add/remove causality, repeated re-tests	2.0
Broker build	Two-part batch-logon broker; .NET enumeration; netsh apply/revert + bug fixes	4.5
DHCP app	User-mode DHCP self-assign breakthrough — built & signed, then set aside as the fallback	4.0
Productionize	ps2exe, code-signing / enterprise-CA PKI, SCCM packaging + deploy + real-hardware test	5.0
JEA	Least-privilege delegation design analysis	2.0
Root cause	<code>ConsentPromptBehaviorUser</code> + <code>NC_LanChangeProperties</code> GPO test on a hardened VM	2.5
Late-night debugging	Lab breakage, DHCP/APIPA recovery, token & registry-hive wrangling — the stuff that only breaks at 1 a.m.	4.0
Testing	Extensive cross-VM + real-hardware validation and re-runs	3.0
Tool teardown	Simple IP Config decompile + analysis	1.5
Meetings & coordination	Reviews, back-and-forth, decisions	2.5
Documentation & write-up	Brain notes, dossier, PDF, publishing	3.0
Total (estimated)		45.0

THE ONE-PARAGRAPH ANSWER

The blocker was **never** the Network Configuration Operators (NCO) group, `netsh`, or the "Network Connections" Group Policy node. It was **UAC token**

filtering, which depends on the logon type — and on hardened boxes it was compounded by a single registry value,

`ConsentPromptBehaviorUser = 0` ("automatically deny standard-user elevation").

NCO *does* grant the IP-change right, but UAC **strips NCO out of interactive logon tokens**. It survives intact in **network and batch** logons. So the same account that fails at the desktop succeeds over SSH or from a scheduled task. The fix on the affected machine was

`ConsentPromptBehaviorUser 0 → 3`. Three viable app architectures fall out of this: a **batch-logon broker**, a **user-mode DHCP self-assign**, and **JEA**.

01 The problem

Field technicians — non-technical staff, deliberately given **zero local admin** — need to set a laptop's IPv4 to talk to network equipment on an isolated bench link. Privileged tasks are meant to use a separate **xID** account, which is *also* not a local admin. The target image is Windows 11 25H2 under a Microsoft/CIS-style hardening baseline. The tool is `IPChanger`, a PowerShell WinForms app compiled with `ps2exe`.

The obvious answer — put the xID in **Network Configuration Operators**, the built-in group Microsoft ships precisely so non-admins can change TCP/IP — didn't work interactively. Understanding *why* took the whole investigation below.

02 Core mechanism — logon type decides everything

NCO's SID (`S-1-5-32-556`) sits on UAC's list of "administrator-equivalent" groups, alongside Backup Operators and Power Users. When UAC builds a *filtered* token, those groups are set to **deny-only** or dropped entirely. Whether filtering happens is decided by the **logon type**:

Interactive logon

type 2 — console, RDP,
`runas/CreateProcessWithLogonW`

Network & batch logon

type 3 — SSH/SMB/PS-remoting · type 4 —
scheduled task

UAC builds a filtered token. NCO is **deny-only / absent** — the app logged `ncoInToken=False`.

✗ **ncpa.cpl & netsh both fail**

No UAC filtering. `whoami /groups` shows NCO as an **"Enabled group"**.

✓ **netsh changes the IP, exit 0**

This single fact explains everything downstream: every early "it works!" result was measured over SSH (a network logon), and John's original desktop failure was the *same* root cause — not a separate `ncpa.cpl` quirk.

03 Capability matrix

Measured on W104 as `x_Winston` (NCO member) over an unfiltered logon. The takeaways drove the app rewrite: use `netsh` for addresses, the registry for DNS, and **.NET for enumeration** — never CIM.

OPERATION	RESULT	NOTES
<code>netsh interface ipv4 set/add/delete address</code>	WORKS	Applies live; the app's apply/revert path
<code>netsh interface ipv4 delete route</code>	WORKS	Clears the default route a gateway leaves behind
Tcpip <code>NameServer</code> registry write	WORKS	NCO has SetValue on the interface key → DNS applies live
<code>[...]NetworkInformation.NetWorkInterface</code> (.NET)	WORKS	The only enumeration path that survives
<code>netsh ... set/add dnsservers</code>	FAILS	"requires elevation" — use the registry path instead
<code>netsh ... set interface admin=disabled</code>	FAILS	Adapter enable/disable needs real admin
Any CIM/WMI — <code>Get-NetAdapter</code> , <code>Get-NetIPAddress</code>	FAILS	"Cannot connect to CIM server. Access denied" → app must not use these

OPERATION	RESULT	NOTES
<code>ncpa.cpl</code> adapter properties (interactive)	FAILS	Interactive token = NCO stripped (\$02)
UAC over-the-shoulder elevation w/ xID	FAILS	<code>consent.exe</code> error 740 — elevation requires local Administrators ; NCO never qualifies

Control experiment — NCO is provably the cause

Removed `FAFOLAB\NetOps` from the local NCO group → `netsh set address` flipped to "requires elevation" and the registry write to "access not allowed." Re-adding NetOps restored both **instantly, no reboot** (each logon builds a fresh token). The capability is granted by NCO membership itself.

04 Every architecture we tried

1 `-Verb RunAs` **self-elevation**

DEAD END

The app's original model. Elevation requires membership in local Administrators; a least-privilege xID has none, so this can never work.

2 **Credential relaunch** — `Start-Process -Credential` / `CreateProcessWithLogonW`

DEAD END

Prompts for the xID and relaunches as it — but that is an **interactive** logon, so NCO is stripped and the app loops on the prompt (`ncoInToken=False`).

3 `LogonUser(NETWORK)` + `ImpersonateLoggedOnUser`

PARTIAL

Needs no privilege, but impersonation sets the *thread* token; a spawned `netsh` inherits the *process* token. Only helps for in-process work (the registry DNS write did apply this way).

4 **Batch-logon broker**

WORKS

GUI runs as the normal user. On apply: prompt once for the xID, drop a request JSON, register a throwaway task `IPChanger-Apply-<stamp>` `/ru xID /rl LIMITED` (a **batch** logon → NCO enabled, and *not* SYSTEM), run it, poll the result, delete the task. Backend re-validates every field. Sole prereq: grant NetOps "Log on as a batch job" via GPO. Verified end-to-end.

5 User-mode DHCP self-assign — shipped as `IPProvisioner`

WORKS

Zero privilege: a tiny user-mode DHCP server answers **only the selected NIC's own MAC**, and the SYSTEM DHCP Client service applies the address. Egress pinned via `IP_UNICAST_IF`. No admin, no NCO, no batch grant. Isolated-link use only (it's a rogue DHCP server on a live net); yields a held lease, not a static. Signed & SCCM-deployed.

6 JEA — Just Enough Administration (*recommended enterprise answer*)

PROPOSED

A `RestrictedRemoteServer` endpoint exposing only `Set-TechnicianIP`, running as a per-session virtual account with `RunAsVirtualAccountGroups=@('Network Configuration Operators')`. Loopback PS-remoting is a network logon → NCO unfiltered → it works, fully audited. Hurdle: needs WinRM, which the hardening baseline disables (a security conversation, but this is Microsoft's sanctioned least-privilege model).

05 The GPO red herring — `NC_LanChangeProperties`

A fair question came up: could the *User Config* → *Administrative Templates* → *Network* → *Network Connections* policies be an independent cause? On Windows 11 the only policy in that node whose text names NCO is **NC_LanChangeProperties** ("Prohibit access to properties of components of a LAN connection") — Microsoft says enabling it "prohibits Network Configuration Operators from accessing connection components." We tested it directly.

CONDITION (AS X_WINSTON, NCO ENABLED)

NETSH SET ADDRESS

SOURCE=STATIC

Policy **not set**

exit 0 – applied

Policy = 1, planted in the live hive & **confirmed visible in session**

exit 0 – applied

Identical. The policy governs only the `ncpa.cpl` GUI "Properties" button (netshell); `netsh.exe` and every scripted path ignore it. We also confirmed the hardening baseline never sets it, and a standard user can't self-set it (`HKCU\Software\Policies` is admin/GPO-only). **The Network Connections GPO node is ruled out.**

06 Root cause & the real fix

On the affected machine, standard-user elevation was configured to fail silently. Instead of a credential prompt, users got an error and nothing else — the classic symptom of:

```
# HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System
ConsentPromptBehaviorUser = 0 ← "Automatically deny elevation requests"
                             → 3 ← "Prompt for credentials" (Windows default) = FIXED

# GPO: Computer Config → ... → Security Options →
# "User Account Control: Behavior of the elevation prompt for standard users"
```

Why 0 → 3 fixed it

At 0, the standard-user elevation path is auto-denied before NCO can even be evaluated — an error with no prompt. At 3 (or 1, prompt on the secure desktop) the flow proceeds and the NCO-backed change resolves. Note the tension: **CIS/STIG baselines recommend 0**, so a hardened gold image ships pre-broken for this workflow. In the lab baseline we moved 0 → 1 and set `TypeOfAdminApprovalMode 2 → 1` (Administrator Protection off, classic Admin Approval Mode).

07 Field notes & gotchas

- › **netsh static is additive.** Setting a static address keeps the DHCP lease *and* adds the manual one. `set source=dhcp` errors when DHCP is already on — the correct revert is `delete address`.
- › **Gateways leave a route.** Setting a gateway leaves a default route after the address is deleted. Either skip the gateway (same-subnet gear needs none) or `delete route` explicitly.
- › **SeBatchLogonRight is stripped by the baseline.** Only Administrators/Backup Operators/Perf Log Users get it. The broker needs NetOps granted "Log on as a batch job" via GPO — this was the `267011` "task never ran" failure.
- › **CIM/WMI is denied to NCO.** Enumerate adapters with .NET `NetworkInterface`, never `Get-NetAdapter`.
- › **Testing hazard.** `netsh add/set address` on a machine's *only* DHCP NIC can disable DHCP → APIPA (`169.254.x`) → cut remote access. Always test on a second NIC; recover via the hypervisor console (`... set address source=dhcp`).
- › **Elevation ≠ NCO.** A non-admin can never UAC-elevate, full stop — only local Administrators can. NCO grants the *capability*, not the ability to elevate.

08 Recommendation

OPTION	PRIVILEGE NEEDED	SCOPE	TRADE-OFF
JEA (primary)	None (virtual account = NCO)	Anywhere	Enterprise-correct & audited; needs loopback WinRM enabled
Batch-logon broker	Batch-logon right for NetOps	Anywhere, incl. live LAN	True static; needs one GPO grant
DHCP self-assign (fallback)	None at all	Isolated links only	Ships today, zero rights; lease not static, "creative" to explain to security

Suggested path: pilot JEA as the primary, least-privilege delegation (the `IPProvisioner` GUI becomes its friendly client). Keep the signed DHCP self-assign

EXE as the fallback for images where WinRM can't be enabled. The broker is the general-purpose middle option when a true static on a live network is required.

09 Addendum — the tool put forward as a counter-example

"Simple IP Config 2.8 works every time in dev." We pulled it apart to see whether it *beats* the constraint or just *avoids* it. It's an Autolt v3 (EA06) compiled, unsigned executable; we extracted its 234 KB script. It confirms the findings above rather than contradicting them.

PROPERTY	FINDING
Binary	Autolt v3 (EA06) compiled exe · 965 KB · <code>ProductVersion 3.3.14.2</code> = the Autolt runtime, not the app
Signature	UNSIGNED — no Authenticode; wouldn't pass a code-signing / SCCM baseline
Privilege	<code>#requireadmin</code> (source line 1) — self-elevates via UAC at launch (manifest <code>asInvoker</code> + runtime relaunch)
Applies IP via	<code>netsh interface ip set address / set·add·delete dns</code> — shells out to <code>cmd</code> , hidden
Enumerates via	WMI <code>root\cimv2</code> + registry (<code>EnabledDHCP</code>)
Interface	GUI only — no command line, can't be scripted or driven headlessly
App-control / EDR	BLOCKED — unsigned with no trusted publisher, so Airlock allowlisting denies it by default (as would WDAC / AppLocker); Autolt is a packer class EDR is specifically tuned to flag

Where it needs admin — and where it doesn't

STEP	NEEDS ADMIN?	ON A HARDENED, NON-ADMIN BOX
Launch / elevation (<code>#requireadmin</code>)	YES	UAC admin prompt → auto-denied under <code>ConsentPromptBehaviorUser=0</code> ; NCO can't satisfy it (consent 740)
Adapter enumeration (WMI)	YES	"Cannot connect to CIM server. Access denied" → empty list → its own <i>"problem retrieving the adapters"</i> error
Apply the address (<code>netsh</code>)	NO	The one step that works under a non-admin NCO <i>network/batch</i> token — exactly what our broker & JEA designs use

How it elevates — we looked for a **SYSTEM** broker; there isn't one

A reasonable suspicion was that it smuggles privilege via a scheduled task or a **SYSTEM** helper. It doesn't. The entire 234 KB script spawns exactly **one** worker process:

```
$IPID = Run( @ComSpec & " /k " & $cmd, "", @SW_HIDE, ... ) ; $cmd = netsh interface
```

That is a plain `Run()` — **no username, no password, no "runas" verb** — so the child `cmd` / `netsh` simply **inherits the app's own token**. The app is elevated for one reason only: `#requireadmin` on line 1, which relaunches the whole GUI at high integrity. So `netsh` runs as the **logged-on admin user, not SYSTEM**. The async queue wrapped around it (`_AsyncNewCmd`) is just a stdout poller so the window doesn't freeze — not a privilege path.

ESCALATION PRIMITIVE SEARCHED FOR	PRESENT?
Scheduled task / Task Scheduler COM (<code>schtasks</code> , <code>ITaskService</code>)	absent
Windows service (<code>sc create</code> , <code>CreateService</code>)	absent
<code>CreateProcessAsUser</code> / <code>CreateProcessWithTokenW</code> / <code>LogonUser</code>	absent
Run-as-another-user (<code>RunAs()</code> , <code>"runas"</code> verb) / <code>psexec</code> / named-pipe broker	absent

SYSTEM / LocalSystem token acquisition

absent

The token APIs that *do* appear in the binary (`AdjustTokenPrivileges` , `ImpersonateSelf` , `SeDebugPrivilege`) are stock Autolt library includes (`Security.au3` / `Memory.au3`) dragged in as dependencies and never called on the apply path. Net: **nothing in it can bridge a missing admin token** — there is no SYSTEM fallback, so on a non-admin box it just fails.

VERDICT

It is **approach #1 (self-elevate) in an Autolt wrapper**. "Works every time in dev" means the dev user is a **local admin** (and/or UAC is relaxed), so `#requireadmin` gets a full token and `netsh` runs elevated. Drop the same exe on a non-admin, hardened prod image and it hits the *identical* UAC wall this whole investigation is about — and its WMI enumeration adds a second, independent failure. It never solves the hard part; it assumes admin, which is invisible when everyone in the dev room already has it.

So it isn't a counter-example — it's a **confirmation**. Tellingly, the one part of it that needs no admin is its `netsh` apply path, which is precisely the mechanism our non-admin designs are built on.

And in the production environment it almost certainly never runs at all: an unsigned Autolt EA06 binary is denied by **application allowlisting (Airlock)** before privilege even enters the picture — no trusted publisher, not on the allowlist, and an interpreter/packer profile that EDR is built to flag. "Works every time in dev" is measuring a machine with allowlisting off and the user holding admin — the two conditions prod is designed to remove.